

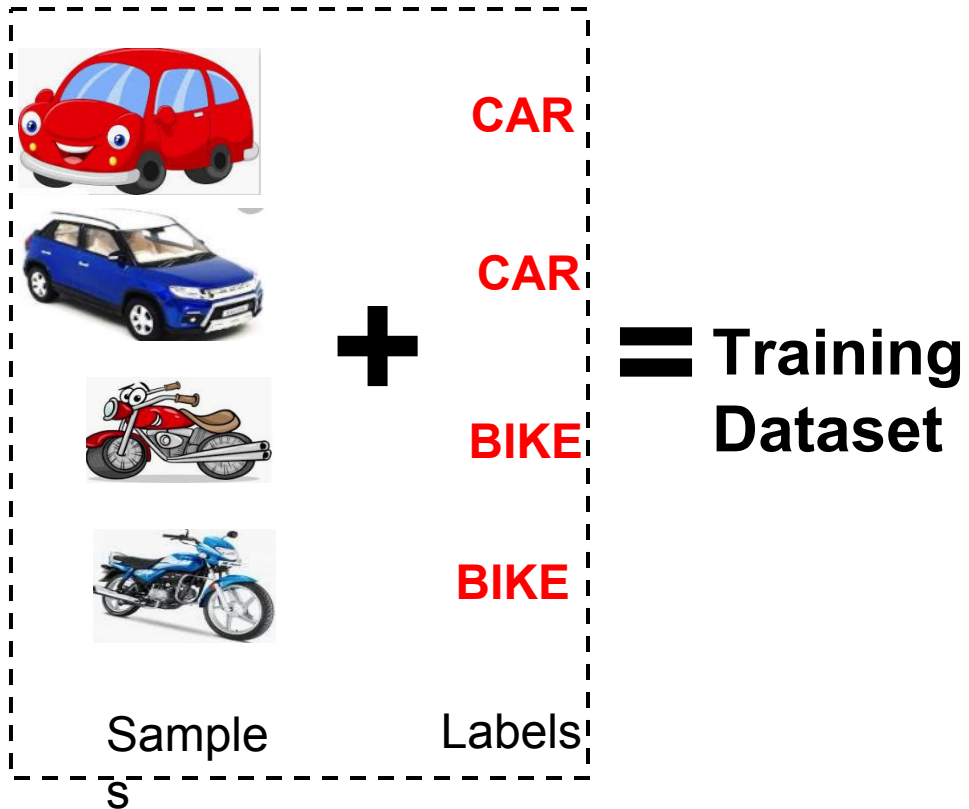
Machine Learning Basics

Introduction to supervised learning

Supervised learning is a type of machine learning where the model is trained on a labeled dataset — that is, data that already contains the correct output (target) for each input.

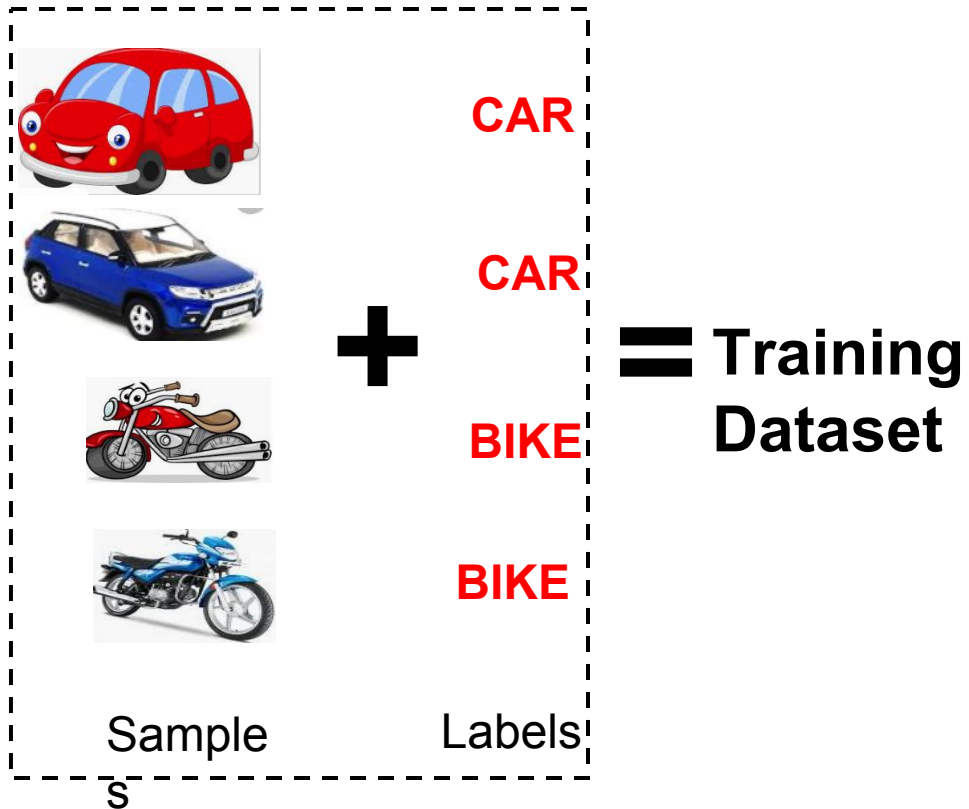
The goal is to learn a mapping from inputs to outputs so that the model can predict the output for new, unseen data.

What is Supervised Learning?



[In supervised learning, we need some thing called a Labelled Training Dataset]

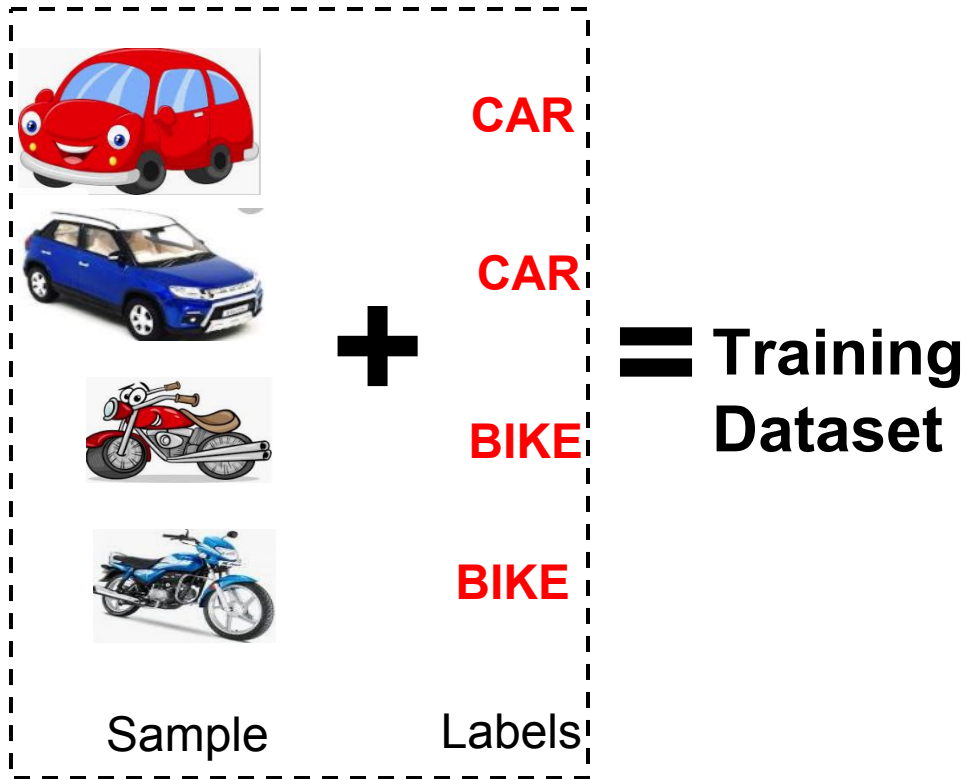
What is Supervised Learning?



$$f(\text{Database}, \text{Sample}) =$$

[Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.]

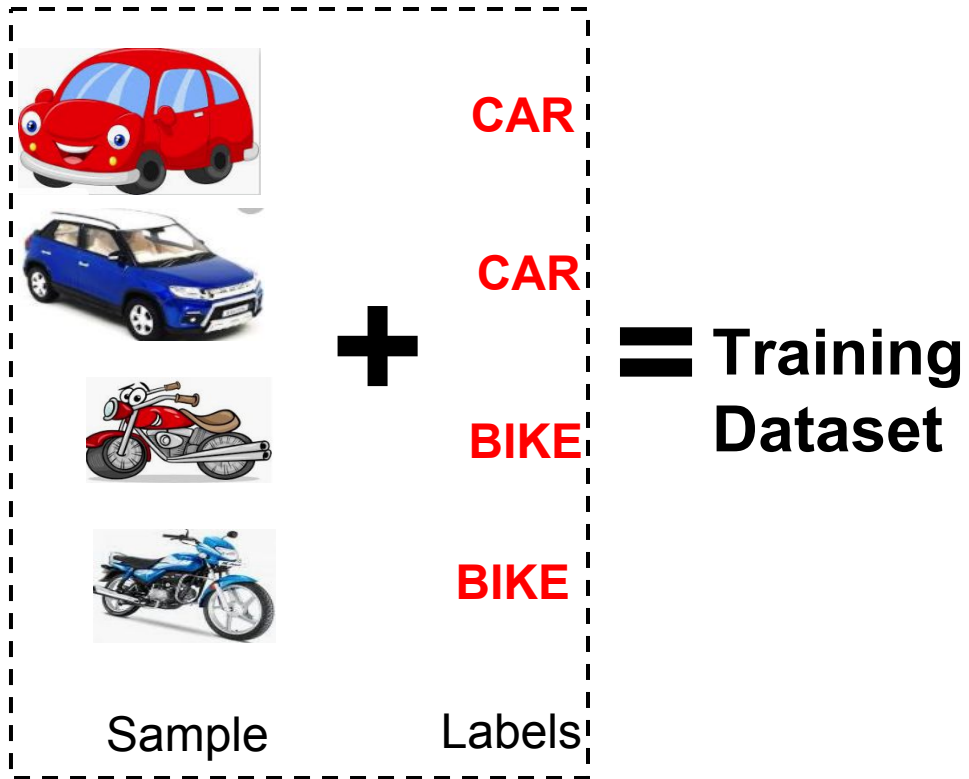
What is Supervised Learning?



$$f(\text{Database}, \text{Sample}) =$$

[Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.]

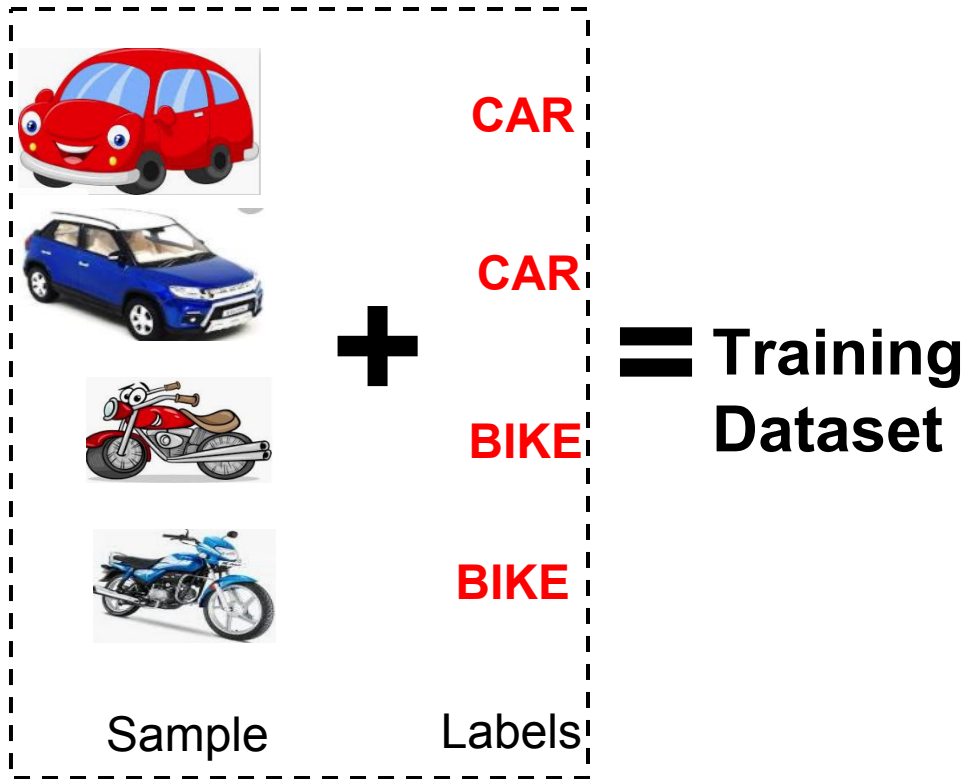
What is Supervised Learning?



$$f(\text{Database}, \text{Sample}) = \text{CAR}$$

[Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.]

What is Supervised Learning?

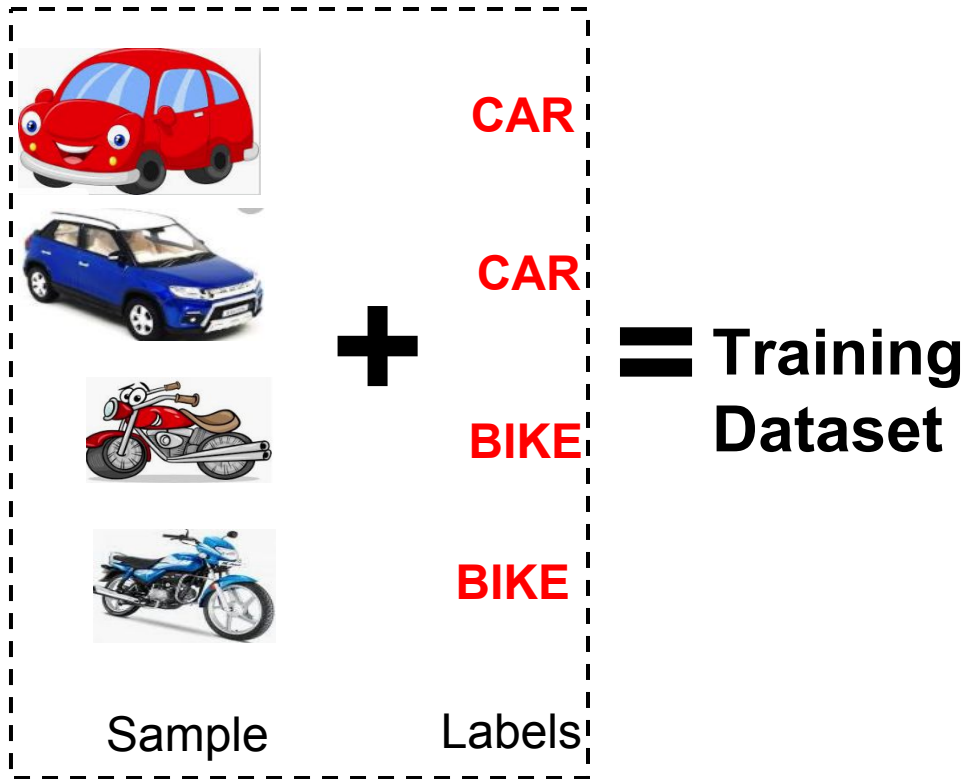


Classification

$$f(\text{Database}, \text{Image of Yellow Car}) = \text{CAR}$$

[If the possible output values of the function are predefined and discrete/categorical, it is called Classification

What is Supervised Learning?



Classification

$$f(\text{Database}, \text{Sample}) = \text{CAR}$$

[Predefined classes means, it will produce output only from the labels defined in the dataset. For example, even if we input a bus, it will produce either CAR or BIKE]

Types of Supervised Learning

There are two main types:

1.Classification – The output variable is **categorical** (discrete labels).

Example: Spam or Not Spam, Pass or Fail, Disease or No Disease.

2.Regression – The output variable is **continuous** (numerical value).

Example: Predicting house price, temperature, or sales amount.

Example of Supervised Learning

Let's say you want to train a model to predict whether a student will **pass or fail** based on their **study hours** and **attendance**.

Study Hours	Attendance (%)	Result
2	60	Fail
4	70	Fail
5	80	Pass
7	90	Pass
8	95	Pass

Here:

- Inputs (features): Study Hours, Attendance
- Output (label): Result (Pass or Fail)

You train the model on this labeled data.

When a new student comes with 6 hours study and 85% attendance, the model predicts whether they will **Pass** or **Fail** based on what it learned.

K-Nearest Neighbors (KNN) Algorithm

K-Nearest Neighbors (KNN) is a simple and intuitive supervised learning algorithm used for both **classification** and **regression**, but mostly for **classification**.

It is based on the idea that **similar data points exist close to each other in the feature space**.

Algorithm

When a new data point needs to be classified:

1. **Compute the distance** between this new point and all points in the training dataset (usually using Euclidean distance).

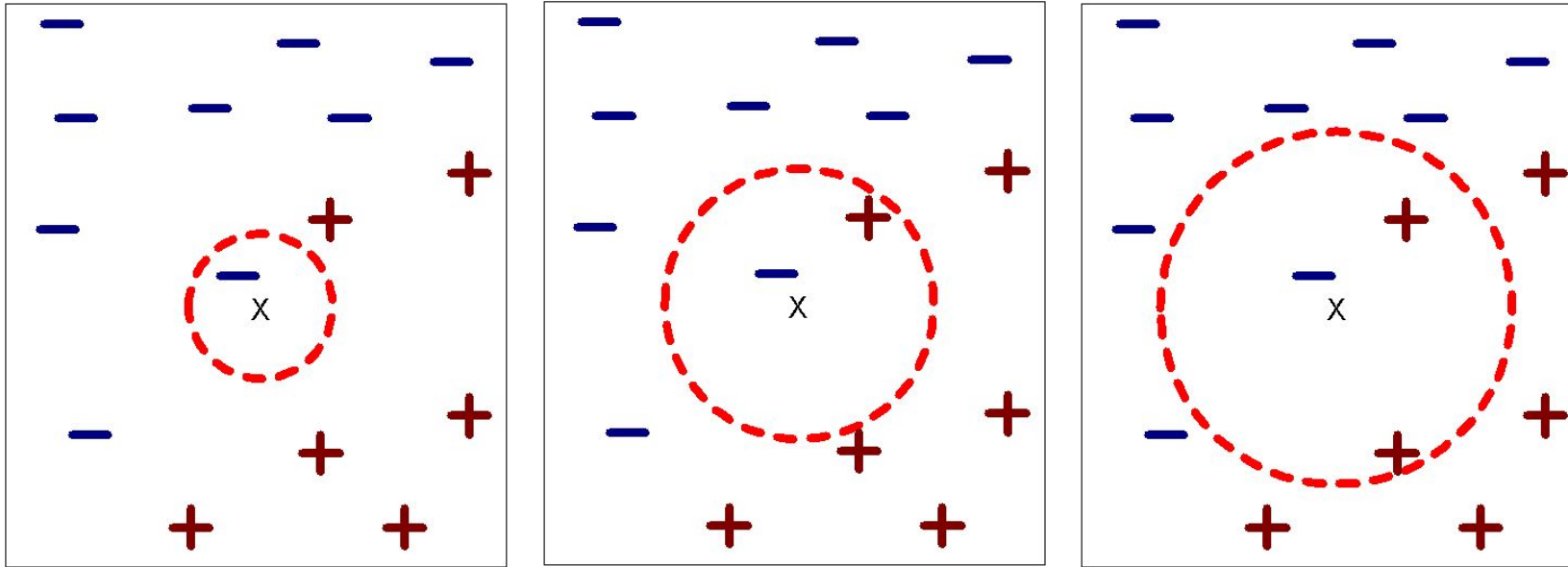
Distance Metric

Most commonly used:

$$\text{Euclidean distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

2. **Select the K nearest neighbors** (the closest data points).
3. **Majority voting** – The class that appears most among the K neighbors becomes the predicted class for the new data point.

Definition of Nearest Neighbor



(a) 1-nearest neighbor

(b) 2-nearest neighbor

(c) 3-nearest neighbor

K -nearest neighbors of a record x are data points that have the k smallest distance to x

Example

🎓 Example: Predicting Student Result (Pass / Fail)

We have data on how many hours students studied and how many classes they attended.

Student	Study Hours (x_1)	Classes Attended (x_2)	Result (y)
A	1	2	Fail
B	2	3	Fail
C	3	4	Pass
D	4	5	Pass
E	5	7	Pass

We want to predict for a **new student (F)**:

$$x_{new} = (3.5, 4)$$

We will use **K = 3**.

Step 1: Compute Euclidean Distance

Formula for Euclidean distance:

$$d(x_{new}, x_i) = \sqrt{(x_{new}^{(1)} - x_i^{(1)})^2 + (x_{new}^{(2)} - x_i^{(2)})^2}$$

Now compute for each training point:

Student	(x_1, x_2)	Result	Distance to (3.5, 4)
A	(1, 2)	Fail	$\sqrt{(3.5 - 1)^2 + (4 - 2)^2} = \sqrt{(2.5)^2 + 2^2} = \sqrt{10.25} = 3.20$
B	(2, 3)	Fail	$\sqrt{(1.5)^2 + (1)^2} = \sqrt{3.25} = 1.80$
C	(3, 4)	Pass	$\sqrt{(0.5)^2 + (0)^2} = \sqrt{0.25} = 0.50$
D	(4, 5)	Pass	$\sqrt{(0.5)^2 + (1)^2} = \sqrt{1.25} = 1.12$
E	(5, 7)	Pass	$\sqrt{(1.5)^2 + (3)^2} = \sqrt{11.25} = 3.35$

Step 2: Find the K Nearest Neighbors

Arrange by distance (ascending):

Rank	Student	Distance	Result
1	C	0.50	Pass
2	D	1.12	Pass
3	B	1.80	Fail
4	A	3.20	Fail
5	E	3.35	Pass

For **K = 3**, we take the **3 nearest neighbors**:

Students **C, D, B**.

Step 3: Majority Voting

Among C, D, and B:

Neighbor	Result
C	Pass
D	Pass
B	Fail

✓ Pass: 2 votes, Fail: 1 vote



Step 4: Predicted Class

$$\hat{y}_{new} = \operatorname{argmax}_{c \in \{Pass, Fail\}} \sum_{i \in N_K} I(y_i = c)$$

$$\hat{y}_{new} = Pass$$

✓ Predicted Result: PASS

Key Observations

- When $K = 3$, result = **Pass**
- If $K = 5$, still **Pass (3 Pass vs 2 Fail)**
- If $K = 1$, only neighbor (3,4) → **Pass**

So, the choice of **K** affects the decision boundary:

- **Small K** → sensitive to noise (overfitting)
- **Large K** → smoother boundary (underfitting)



1 Meaning of K in KNN

- **K** = the number of nearest neighbors to consider when classifying or predicting a new data point.
- It controls how much the model “looks around” to make a decision.

Value of K	Model Behavior
Small K (like 1 or 2)	Sensitive to noise → may overfit
Large K (like 15 or 20)	Too smooth, ignores small local patterns → may underfit

So, **K controls the bias–variance tradeoff.**

**2**

The Bias–Variance Tradeoff

Case	Effect	Model Characteristic
Small K (e.g., K=1)	Low bias, high variance	Fits training data closely, sensitive to outliers
Large K (e.g., K=15)	High bias, low variance	Stable predictions, but might miss class boundaries

So, the **ideal K** balances these two — not too small, not too large.

3 Practical Methods to Choose K

(a) Cross-Validation Method

The most common and robust approach.

1. Split your dataset into **training** and **validation** sets.
2. Try different values of K (say, 1, 3, 5, 7, 9, 11...).
3. For each K, measure the model's **accuracy** on the validation set.
4. Choose the K that gives **highest validation accuracy**.

(b) Rule of Thumb (Heuristic)

A simple starting point:

$$K = \sqrt{N}$$

where N = number of data points in the training set.

For example:

- If $N = 100 \rightarrow K = 10$
- If $N = 400 \rightarrow K = 20$

You can then fine-tune around this value.

(c) Odd Number of K

If you have **binary classification (two classes)**, always prefer **odd K** (e.g., 3, 5, 7) to **avoid ties** in voting.

Important Notes

- Always **normalize or scale** your data before applying KNN, otherwise features with large ranges (like “salary” vs “age”) dominate distance calculations.

Example: use **MinMaxScaler** or **StandardScaler** in sklearn.

- K can vary for different datasets — there’s **no universal best K**.

Advantages of KNN

- Simple to understand and implement.
- No training phase.
- Works well with small datasets.

Disadvantages of KNN

- Computationally expensive for large datasets (distance must be computed for every point).
- Sensitive to irrelevant or scaled features.
- Struggles with high-dimensional data.

Python Code

```
from sklearn.neighbors import KNeighborsClassifier

# Training data
X = [[120, 0], [150, 0], [180, 1], [200, 1]]
y = ['Apple', 'Apple', 'Orange', 'Orange']

# Create and train model
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

# Predict new fruit
prediction = knn.predict([[160, 1]])
print("Predicted fruit:", prediction[0])
```

Practice Question

Dataset

Fruit	Weight (grams) (x_1)	Sugar Content (%) (x_2)	Type (y)
A	120	10	Apple
B	140	8	Apple
C	160	15	Mango
D	170	14	Mango
E	155	11	Apple

Question:

A new fruit has

$$x_{new} = (150, 12)$$

Classify it as **Apple** or **Mango** using the **KNN algorithm** with $K = 3$ and **Euclidean distance**.

Practice Question

Dataset

Patient	Glucose Level (x_1)	BMI (x_2)	Has Diabetes (y)
P1	80	20	No
P2	120	28	No
P3	140	30	Yes
P4	170	35	Yes
P5	200	40	Yes

Question:

A new patient has

$$x_{new} = (150, 33)$$

Use the **K-Nearest Neighbors (KNN)** algorithm with $K = 3$ and **Euclidean distance** to predict whether the patient has diabetes (Yes/No).

What is KNN Regression?

KNN can be used not only for **classification** (categorical output) but also, for **regression**, where the output (target variable) is **continuous (numeric)**.

Instead of voting for the majority class, in regression KNN takes the **average (or weighted average)** of the target values of the nearest neighbors.

2 Example: Predicting House Price

Dataset

House	Size (sq ft) (x)	Price (₹ lakh) (y)
H1	1000	50
H2	1200	60
H3	1500	70
H4	1800	90
H5	2000	100

We want to predict the **price** of a new house:

$$x_{new} = 1600$$

using **K = 3** nearest neighbors.

Step 1: Compute Distances

$$d(x_{new}, x_i) = |x_{new} - x_i|$$

House	Size (x)	Distance	Price (₹ lakh)
H1	1000	600	50
H2	1200	400	60
H3	1500	100	70
H4	1800	200	90
H5	2000	400	100

Step 2: Select K Nearest Neighbors (K = 3)

Closest 3 houses:

- H3 (distance 100, price 70)
 - H4 (distance 200, price 90)
 - H2 (distance 400, price 60)
-

Step 3: Predict Using Mean (Simple KNN Regression)

$$\hat{y}_{new} = \frac{70 + 90 + 60}{3} = \frac{220}{3} = 73.33$$

✓ Predicted House Price = ₹ 73.33 lakh

What is Unsupervised Learning?

Unsupervised learning is a type of **machine learning** where the model is **not given any labeled data**.

That means we don't tell the algorithm what the correct output is — it tries to find patterns, structure, or relationships in the data **on its own**.

For example, if you give a set of customer data (like age, income, and spending habits) to an unsupervised algorithm, it might automatically group similar customers together — **without being told what those groups represent**.

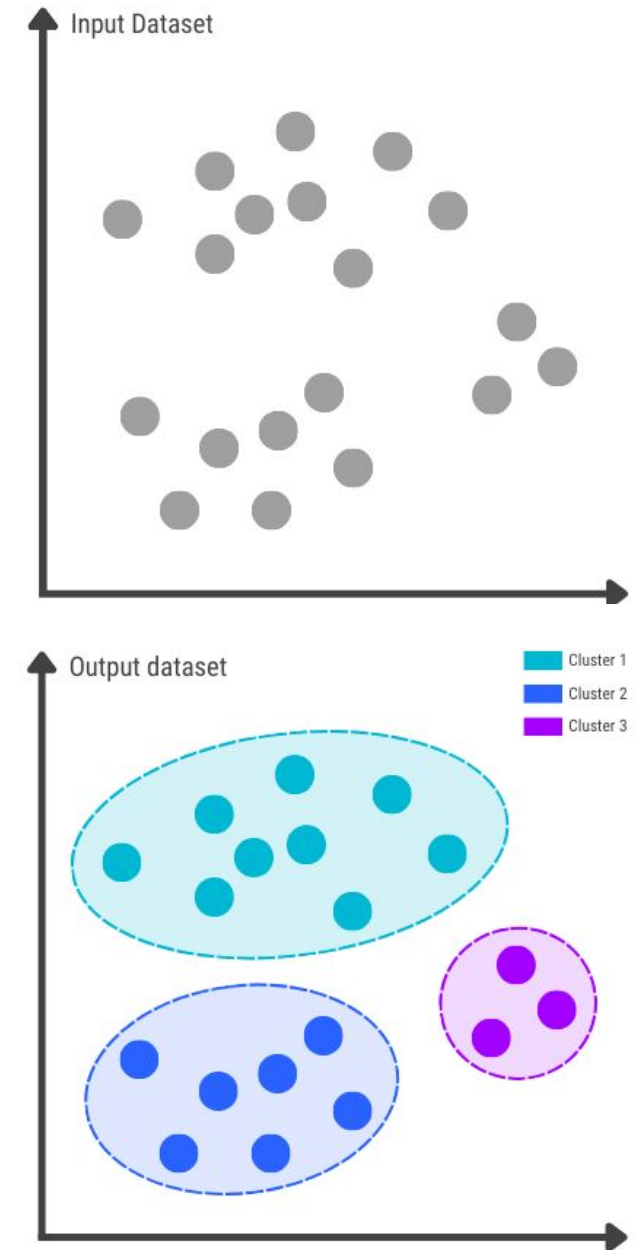
Clustering – The Core of Unsupervised Learning

One of the most common tasks in unsupervised learning is **clustering**.

Definition:

Clustering is the process of dividing a dataset into groups (clusters) so that:

- Data points **within the same cluster** are similar to each other.
- Data points **in different clusters** are as different as possible.



K-Means Clustering

Objective

K-Means aims to partition the dataset into **K clusters**, where **K** is a number chosen by the user.

Each cluster has a **centroid** (center point), and every data point is assigned to the cluster whose centroid is closest to it.

Steps in K-Means Algorithm

- 1. Choose the number of clusters (K)** — e.g., $K = 3$.
- 2. Initialize centroids** — randomly pick K points from the dataset as initial cluster centers.
- 3. Assign points to clusters** — for each data point, find the nearest centroid (using distance measure like Euclidean distance).
- 4. Update centroids** — calculate the new mean position of all points in each cluster to get new centroids.
- 5. Repeat Steps 3–4** until centroids stop changing (i.e., the algorithm converges).

Example

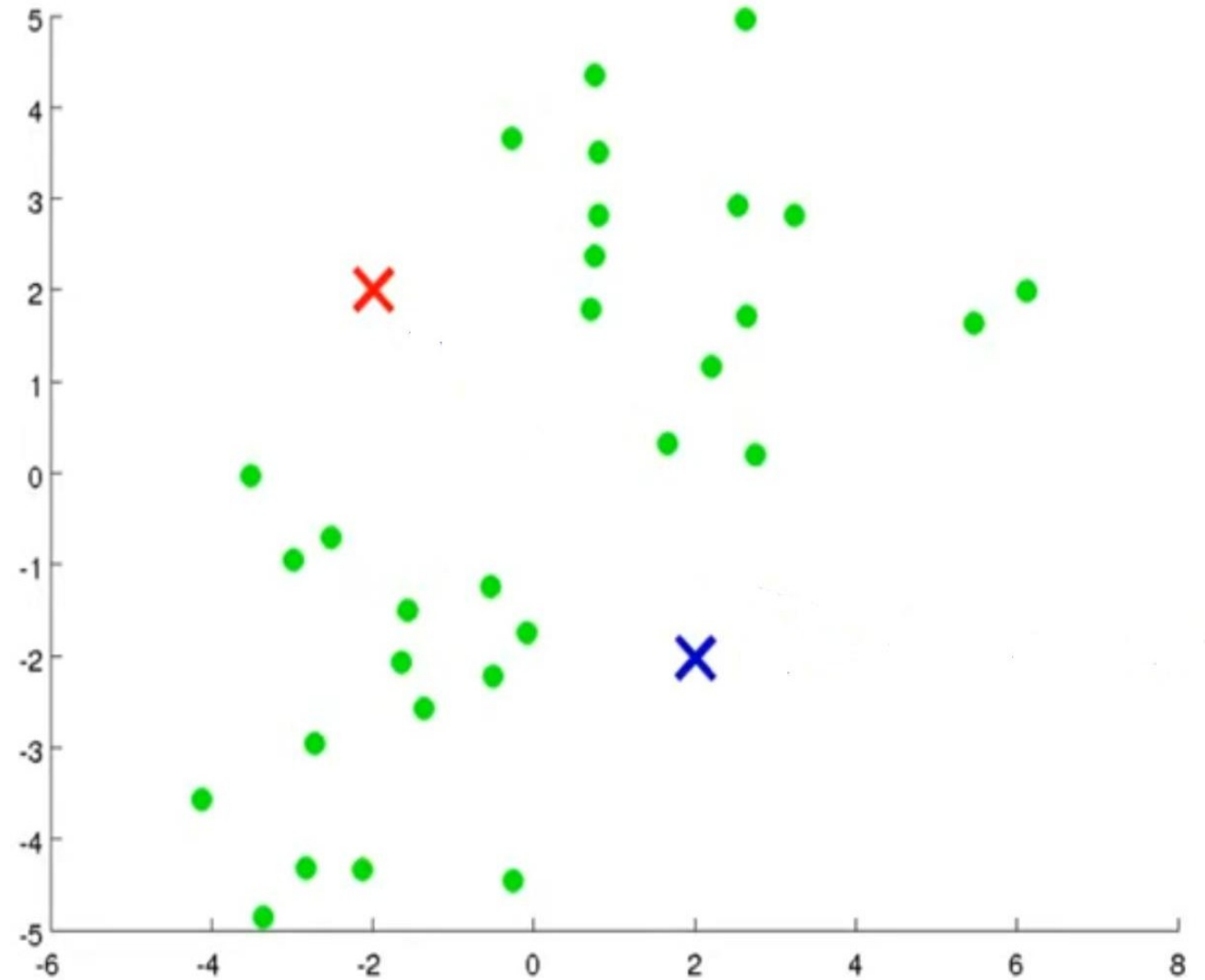
Suppose we have data on students based on their **study hours** and **marks**.

If we apply K-Means with $K = 2$:

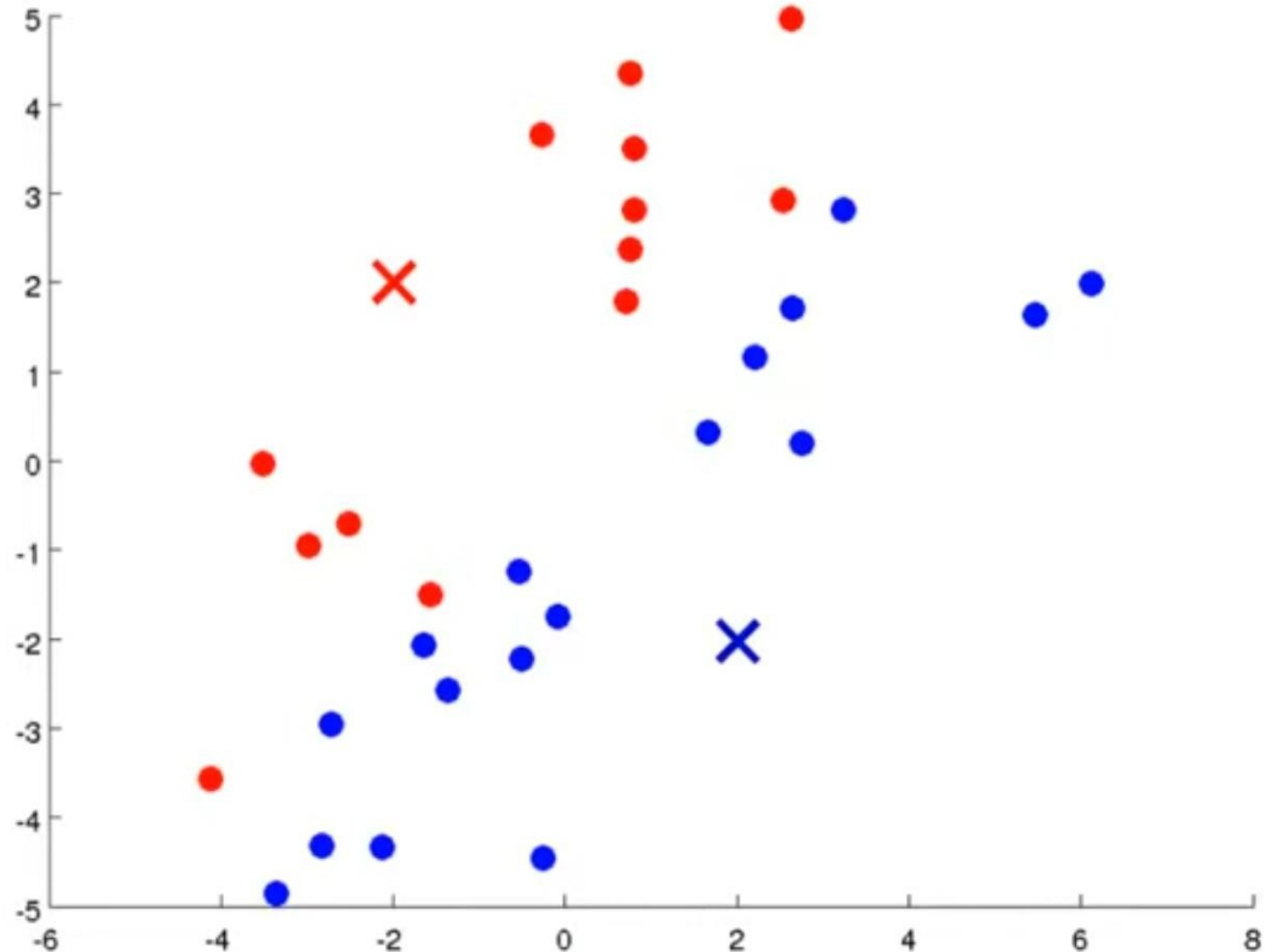
- Cluster 1 might represent “**Low effort students**” (low hours, low marks).
- Cluster 2 might represent “**High effort students**” (high hours, high marks).

The algorithm automatically discovers these patterns without any labels.

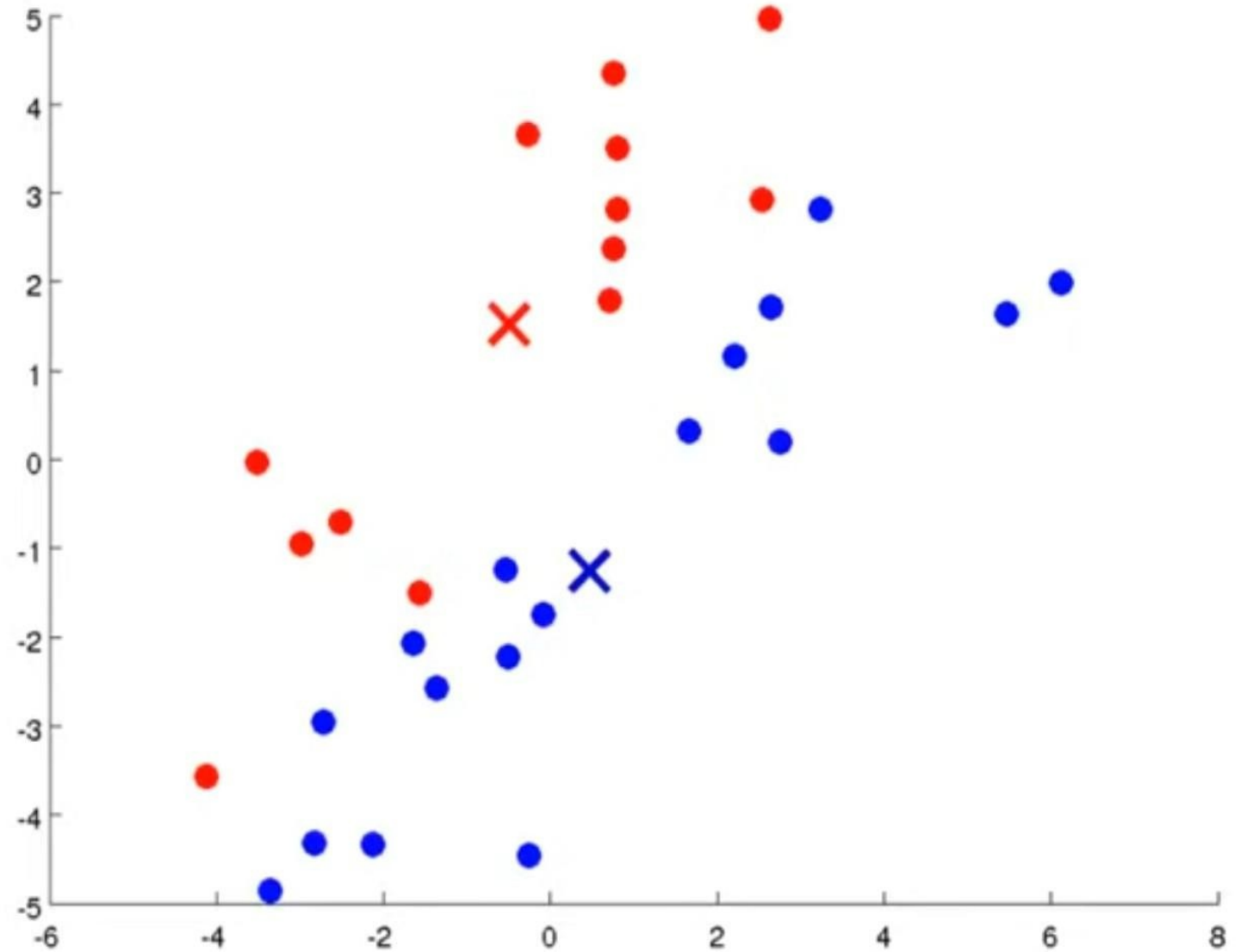
Consider two cluster centers
randomly



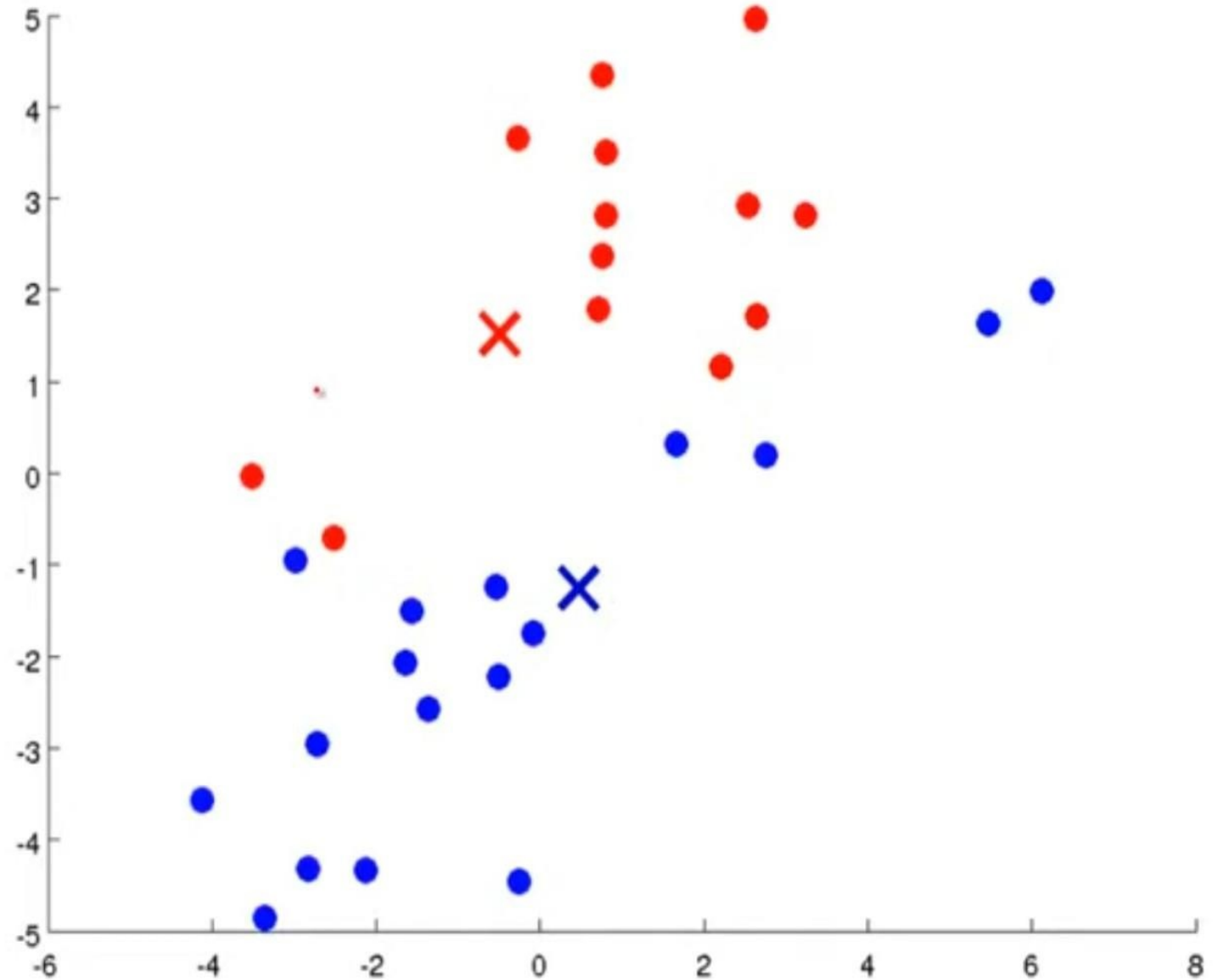
Assign clusters to each
point (shown in same color)
by using minimum distance



Compute mean of each
cluster
and assign it corresponding
new cluster center

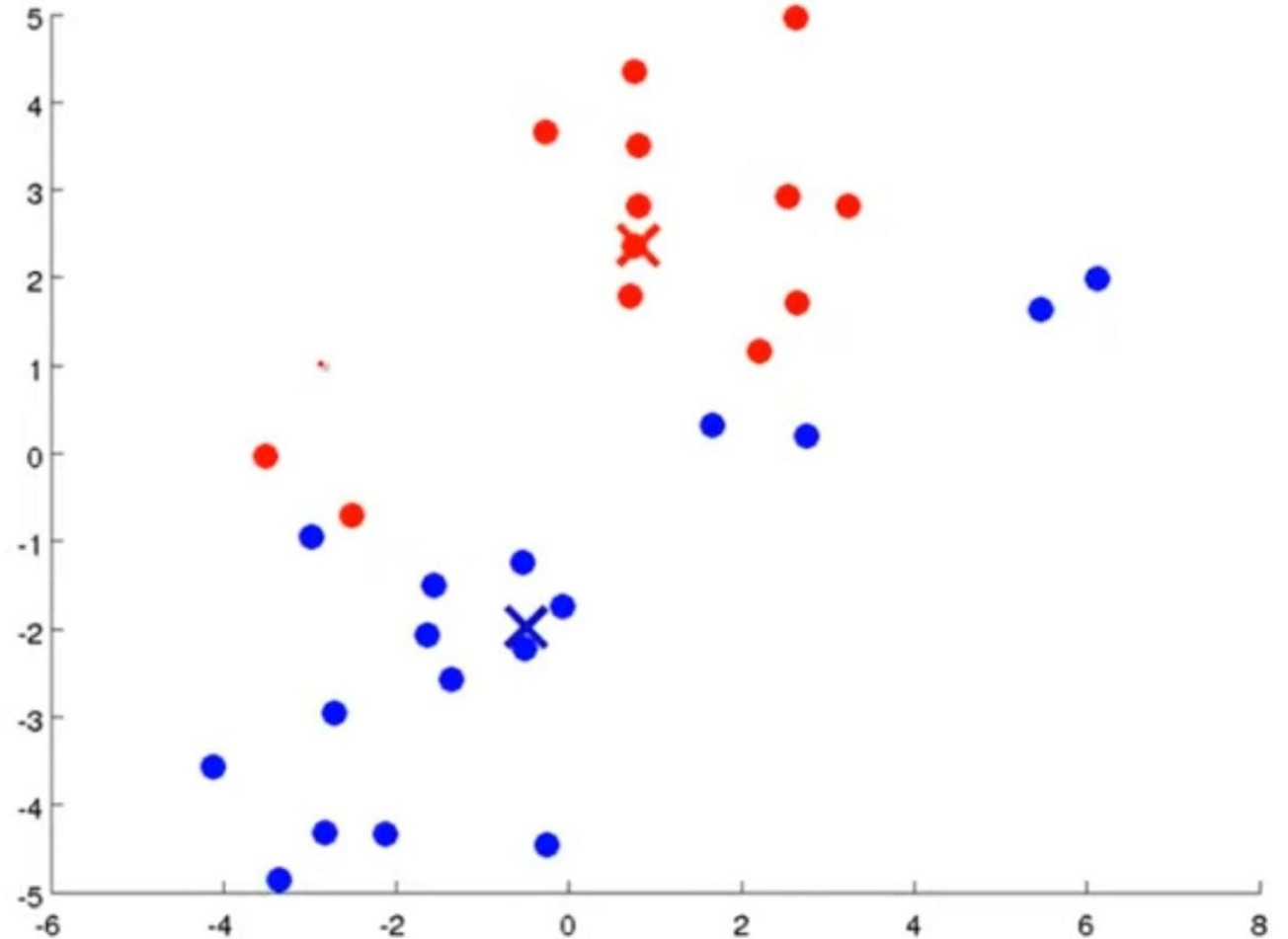


Assign new cluster to each
point (shown in same color)
by using minimum
distance with new
cluster center

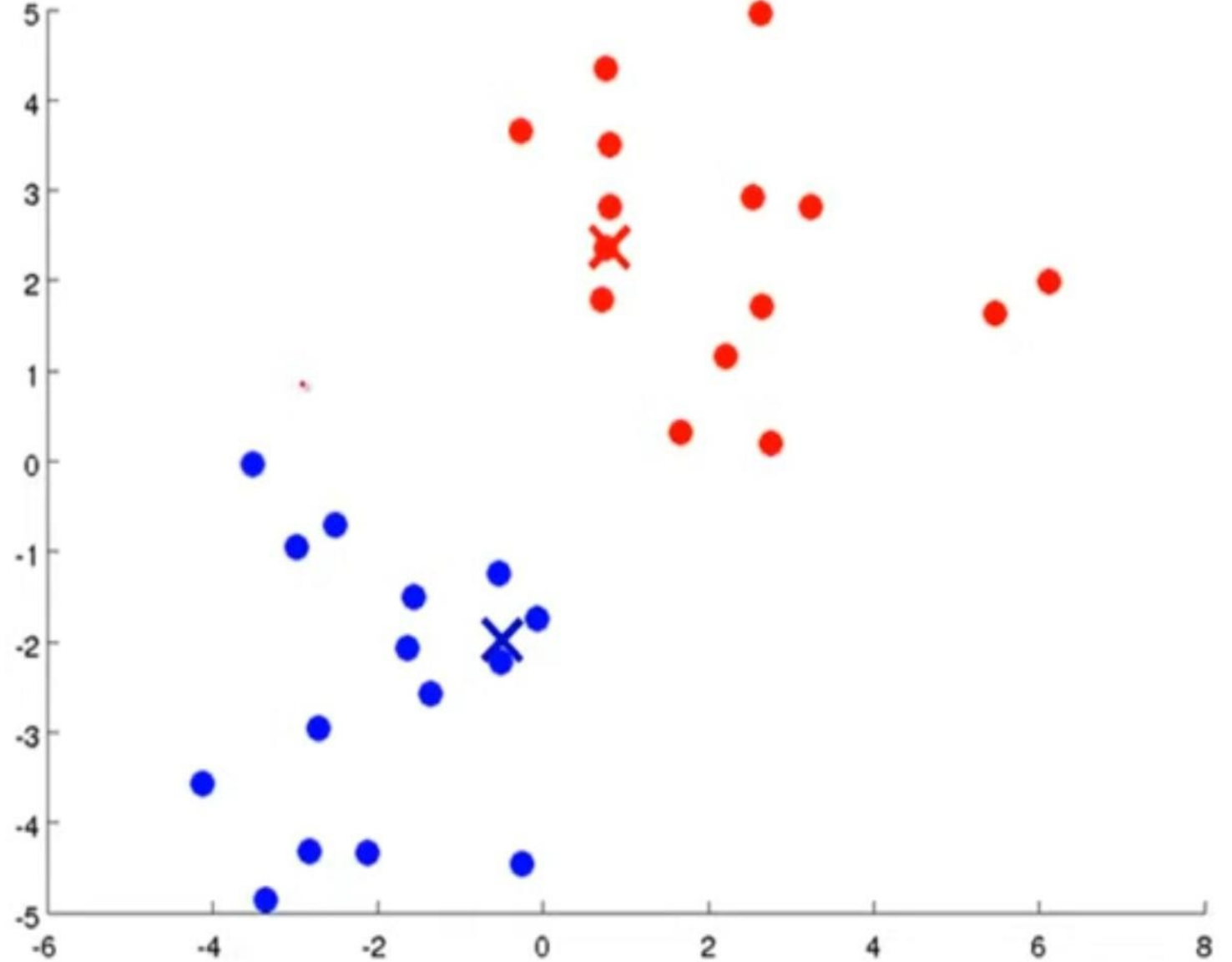


Again compute
mean of each cluster

and assign it corresponding
new cluster center

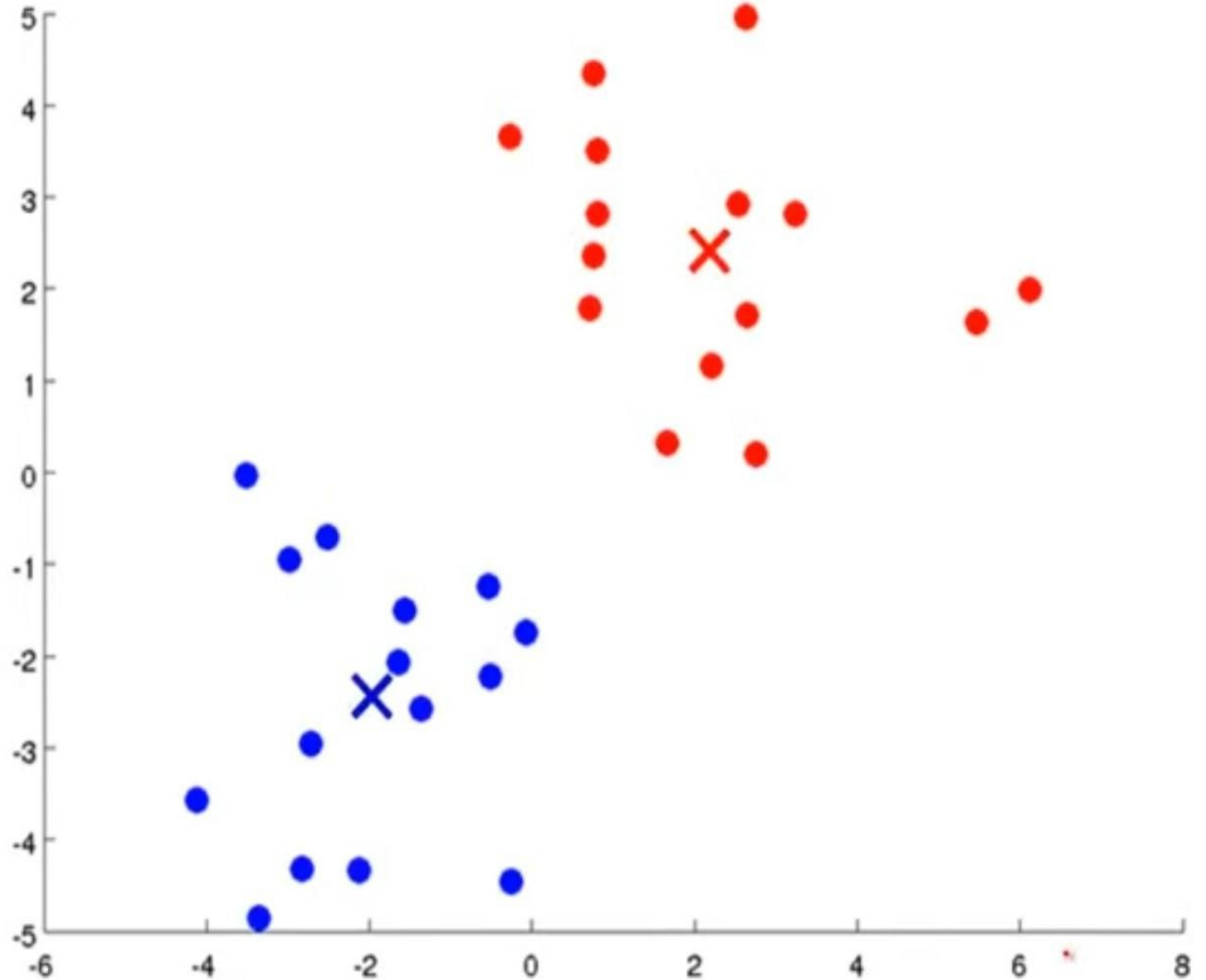


Again, assign new cluster to
each point
(shown in same color) by
using minimum distance
with new cluster center



Again, compute mean
of each cluster and
assign it
corresponding new
cluster center

If, after an iteration, the
**centroid positions remain
the same**, the algorithm has
converged. Then, stop.



Evaluating Model Performance in Clustering

Unlike supervised learning, **we don't have true labels**, so evaluation is based on **cluster quality** rather than accuracy.

Common Evaluation Methods:

1. Inertia (Within-cluster sum of squares):

Measures how close data points are to their cluster center.

- Lower inertia = better clustering.
- Computed as:

$$\text{Inertia} = \sum_{i=1}^k \sum_{x \in C_i} ||x - \mu_i||^2$$

where C_i = cluster i and μ_i = centroid of cluster i.

2. Silhouette Score:

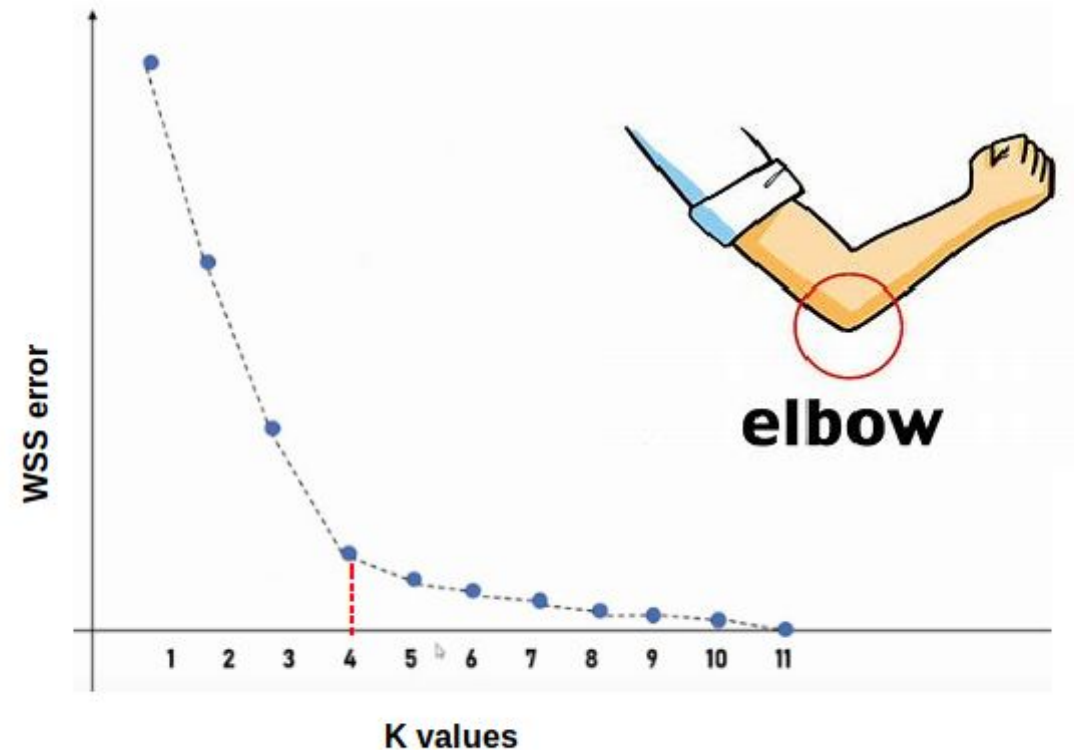
Measures how similar a point is to its own cluster compared to other clusters.

- Value ranges from **-1 to +1**
- Closer to **+1** → good clustering separation.

3. Elbow Method:

Helps to find the optimal value of K by plotting the **inertia vs. number of clusters** and looking for an "elbow point".

Within-Cluster Sum of Squares (WCSS)



Question

Apply **K-Means clustering** (with $K = 2$) to the following small dataset and show **one iteration** of the algorithm manually.

Point	X	Y
A	2	10
B	2	5
C	8	4
D	5	8
E	7	5

Assume initial **centroids** are:

- Cluster 1 (C1): (2, 10)
- Cluster 2 (C2): (5, 8)

Perform **one iteration** of K-Means:

1. Calculate distance of each point to both centroids.
2. Assign each point to the nearest centroid.
3. Compute new centroids for the clusters.

Step 1: Calculate Euclidean Distance

Formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Point	To C1 (2,10)	To C2 (5,8)	Nearest
A(2,10)	0	$\sqrt{[(5-2)^2 + (8-10)^2]} = \sqrt{(9+4)} = \sqrt{13} = 3.61$	C1
B(2,5)	$\sqrt{[(2-2)^2 + (10-5)^2]} = \sqrt{25} = 5$	$\sqrt{[(5-2)^2 + (8-5)^2]} = \sqrt{18} = 4.24$	C2
C(8,4)	$\sqrt{[(8-2)^2 + (4-10)^2]} = \sqrt{(36+36)} = \sqrt{72} = 8.49$	$\sqrt{[(8-5)^2 + (4-8)^2]} = \sqrt{(9+16)} = \sqrt{25} = 5$	C2
D(5,8)	$\sqrt{[(5-2)^2 + (8-10)^2]} = \sqrt{(9+4)} = \sqrt{13} = 3.61$	0	C2
E(7,5)	$\sqrt{[(7-2)^2 + (5-10)^2]} = \sqrt{(25+25)} = \sqrt{50} = 7.07$	$\sqrt{[(7-5)^2 + (5-8)^2]} = \sqrt{(4+9)} = \sqrt{13} = 3.61$	C2

Step 2: Assign Points to Clusters

Cluster	Points
C1	A(2,10)
C2	B(2,5), C(8,4), D(5,8), E(7,5)

Step 3: Compute New Centroids

To find the new centroid, take the **mean of X and Y** values in each cluster.

- **C1:** only point (2,10)
→ New centroid = (2,10)
- **C2:** points B(2,5), C(8,4), D(5,8), E(7,5)

$$\text{Mean of X} = (2 + 8 + 5 + 7) / 4 = 22 / 4 = 5.5$$

$$\text{Mean of Y} = (5 + 4 + 8 + 5) / 4 = 22 / 4 = 5.5$$

$$\rightarrow \text{New centroid} = (5.5, 5.5)$$

✅ Result After First Iteration

Cluster	Points	New Centroid
C1	A(2,10)	(2,10)
C2	B(2,5), C(8,4), D(5,8), E(7,5)	(5.5, 5.5)

If you run further iterations, the centroids will continue updating until they **stabilize** (no change in cluster assignment).

After building your ML model..

- How well is my model doing? Is it a useful model?
- Will training my model on more data improve its performance?
- Do I need to include more features?

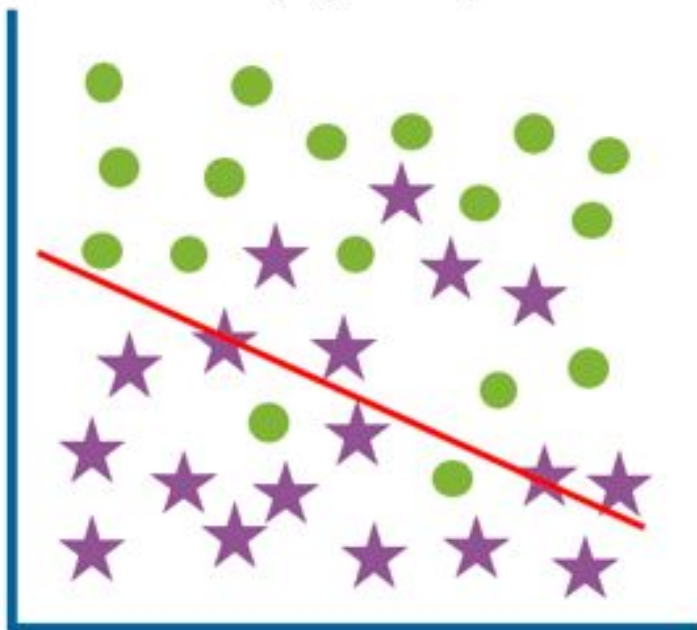
Why Evaluation is necessary for successful model?

- **Overfitting** and **Underfitting** are the two biggest causes for **poor performance** of machine learning algorithms.
- **Overfitting**: Occurs when the Model **performs well** for a particular set of data(Known Data) and may therefore **fail to fit additional data** (Unknown Data) or **predict future observations** reliably.
- Overfitting refers to a model that has **learned the training dataset too well**, including **the statistical noise or random fluctuations** in the training dataset.

- **Underfitting**: Occurs when the model cannot adequately capture the underlying structure of the data.
- **Generalization**: refers to how well the underlying structure of the data, learned by a machine learning model apply to specific examples not seen by the model when it was learning.

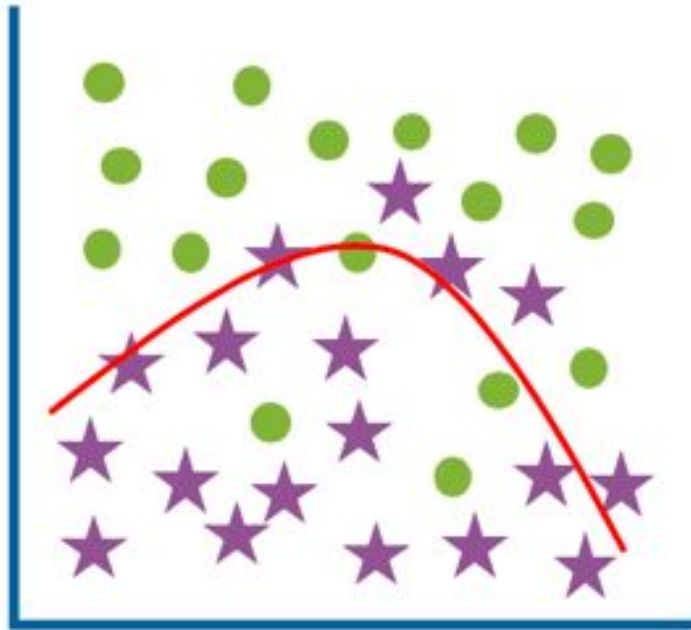
This issue is resolved pretty well with regularization models (LASSO, Ridge Regression, Elastic-Net)

Underfit
(high bias)



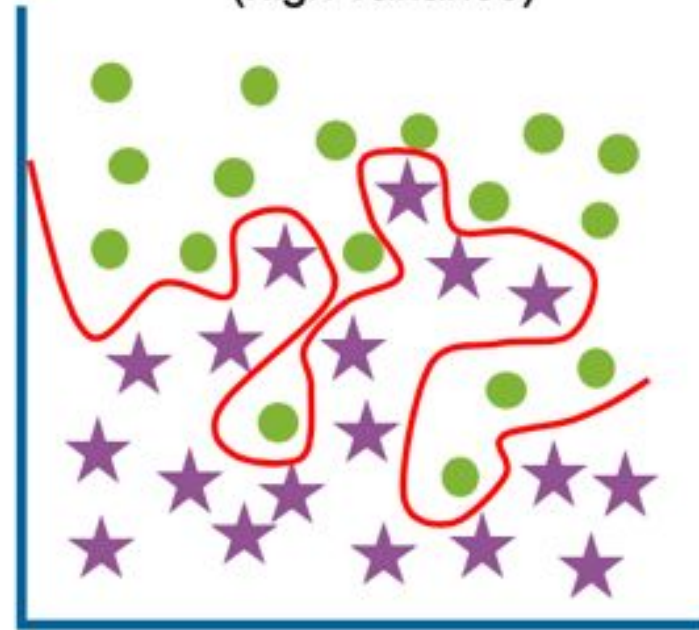
High training error
High test error

Optimum



Low training error
Low test error

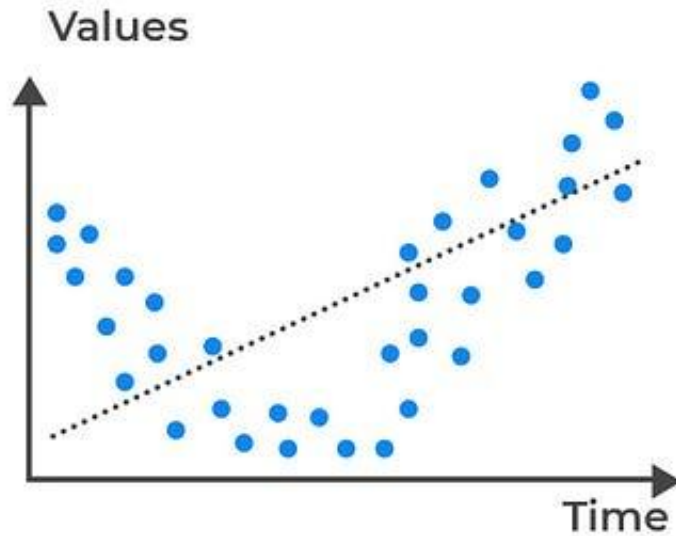
Overfit
(high variance)



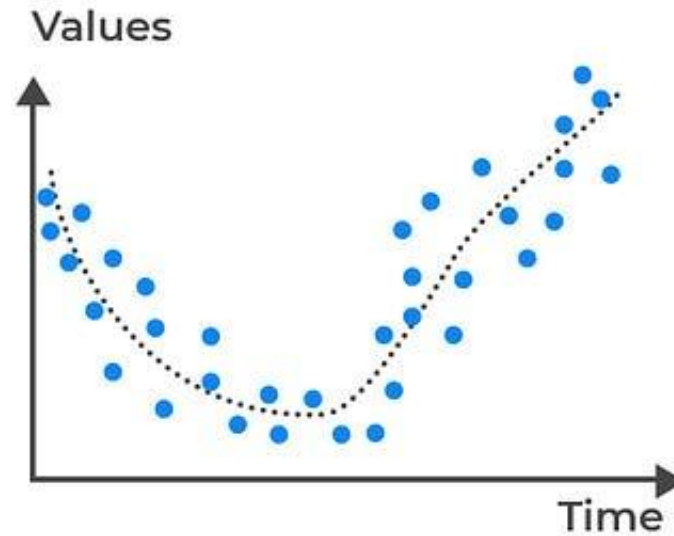
Low training error
High test error



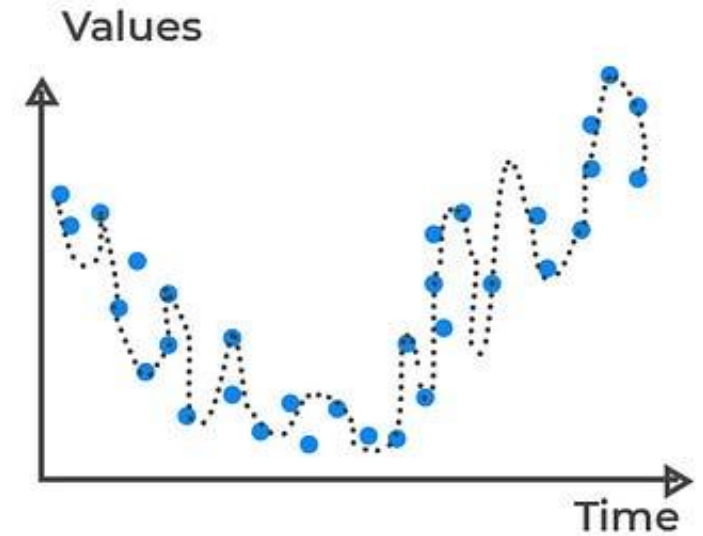
Generalization and Overfitting



Underfitted
(High bias error)



Good Fit/Robust
(Balance between
bias and variance)



Overfitted
(High variance error)

Bias vs Variance

- The ultimate goal of any machine learning model is to learn from examples and generalize some degree of knowledge.
- Some machine learning models provide the framework for generalization by suggesting the underlying structure of that knowledge.

Bias and Variance are two key concepts in Machine Learning that play a crucial role in model selection, performance evaluation and accuracy optimization.

Understanding the differences between these two concepts is crucial in ensuring high-quality machine learning models.

Bias

S

- For example, a linear regression model imposes a framework to learn linear relationships between the information we feed it.
- However, sometimes we provide a model with too much pre-built structure that we limit the model's ability to learn from the examples such as the case where we train a linear model on an exponential dataset.

Bias

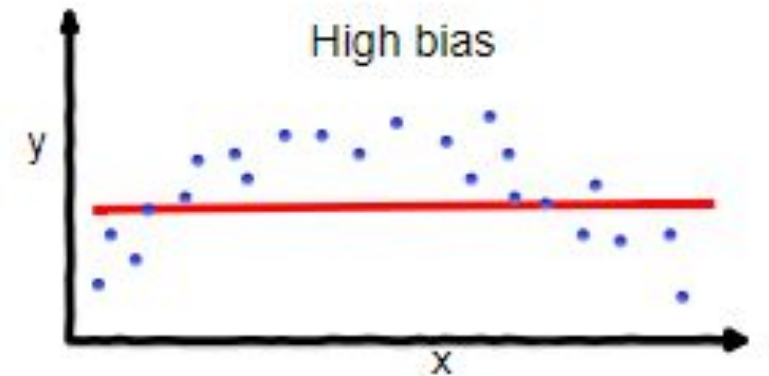
S

- In this case, our model is **biased** by the **pre-imposed structure and relationships**.
- Models with high bias pay little attention to the data (underlying behavior/trends) presented;
- Bias can be introduced by a number of factors, including the choice of features, the type of algorithm used, and the way the model is trained.
- This is also known as **underfitting**.

For Example

A model that only uses a limited number of features may be biased because it does not capture all the important information in the data.

Similarly, an algorithm that is inherently biased, such as a linear regression model, may produce biased predictions because of its underlying assumptions about the relationship between the input and output variables.



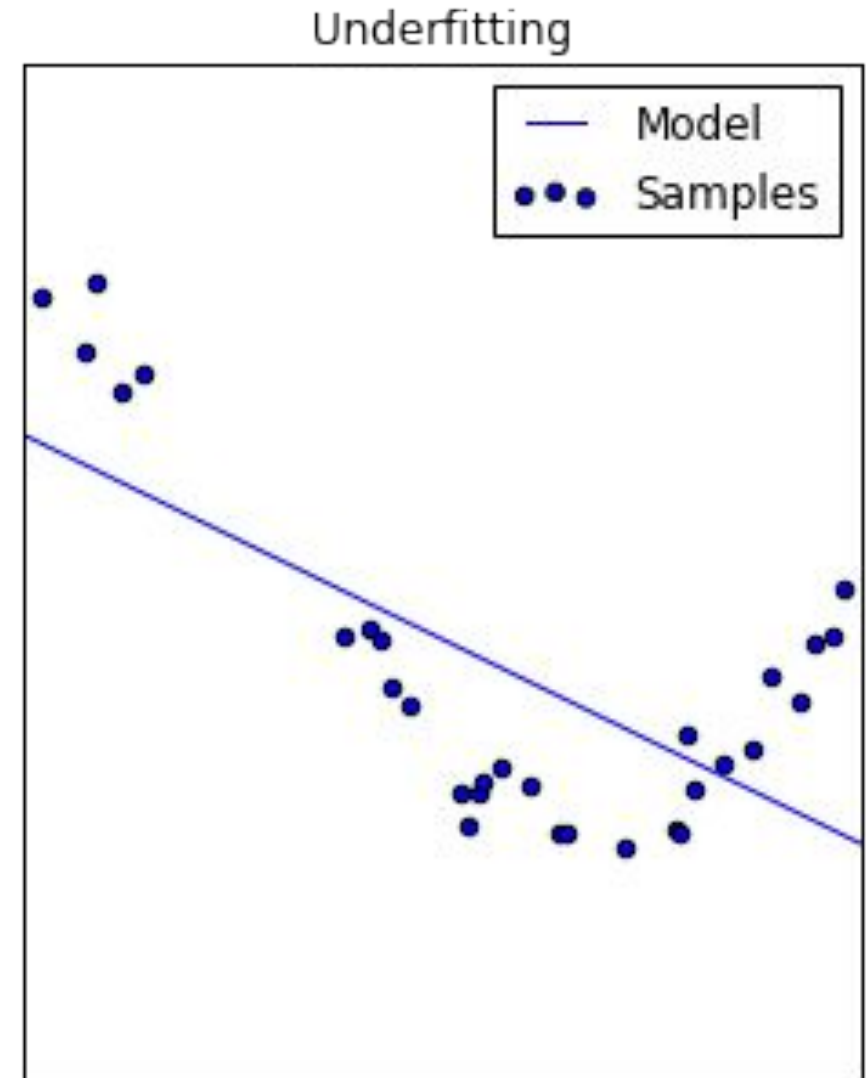
underfitting

<https://everydaycodings.medium.com/difference-between-bias-and-variance-machine-learning-db8f3d43aad9>

Bia

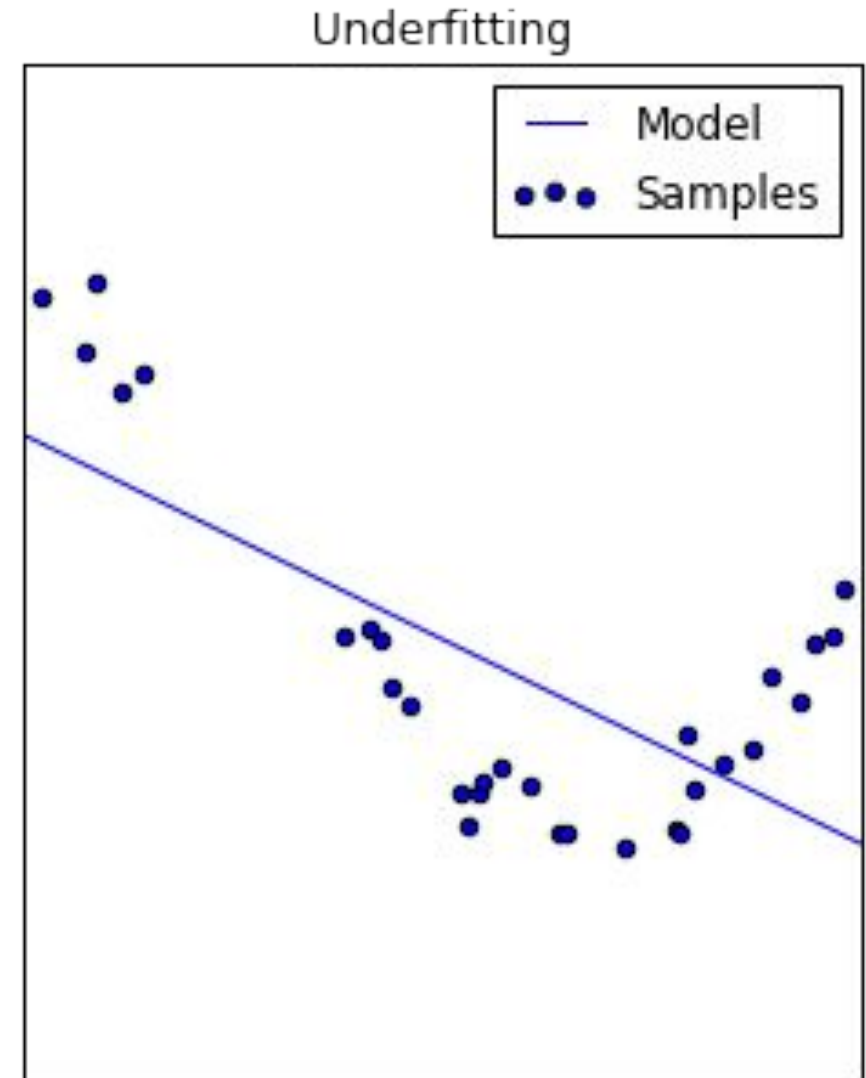
S

- It's also possible to **bias** a model by trying to teach it to perform a task without presenting all of the necessary information.

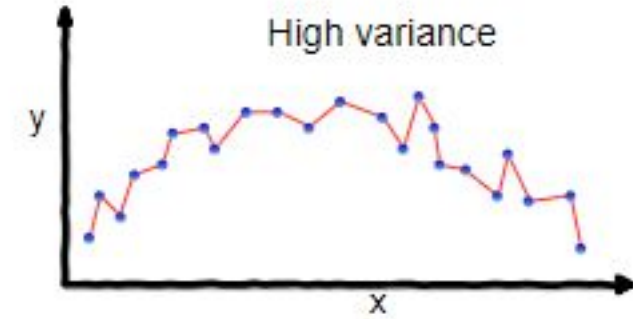


Bias

- If you know the constraints of the model are not biasing the model's performance,
- yet, you're still observed signs of underfitting,
- it's likely that you are not using enough features to train the model.



Variance

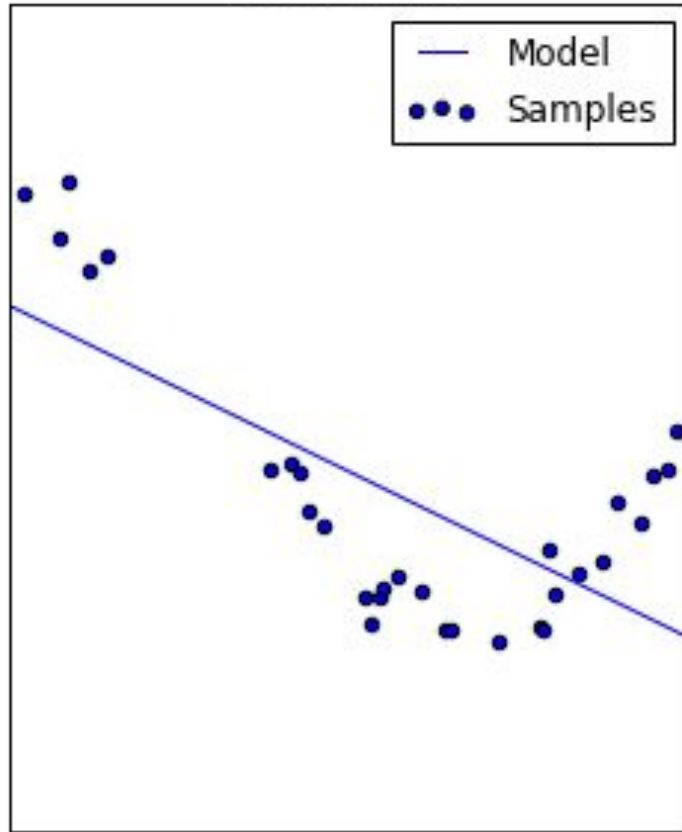


- On the **other extreme**, sometimes when we train our model in such a way that it **learns too much** from the **training data**.
- That is, our **model captures the noise** in the data in addition to the signal.
- This can cause **wild fluctuations** in the model that does **not represent the true trend**;
 - in this case, we say that the model has **high variance**.

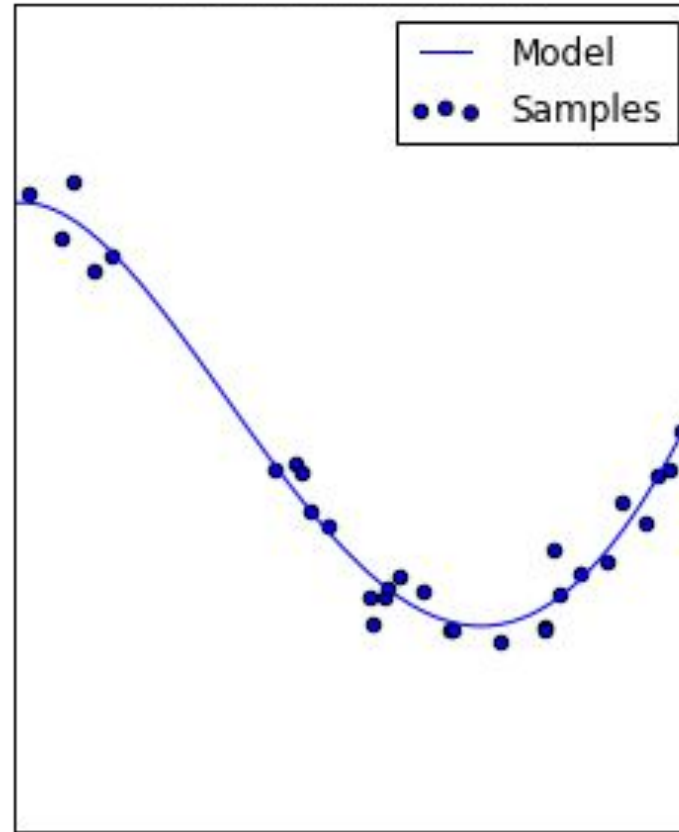
producing high accuracy on the training data but poor accuracy on new data.

Bias vs Variance

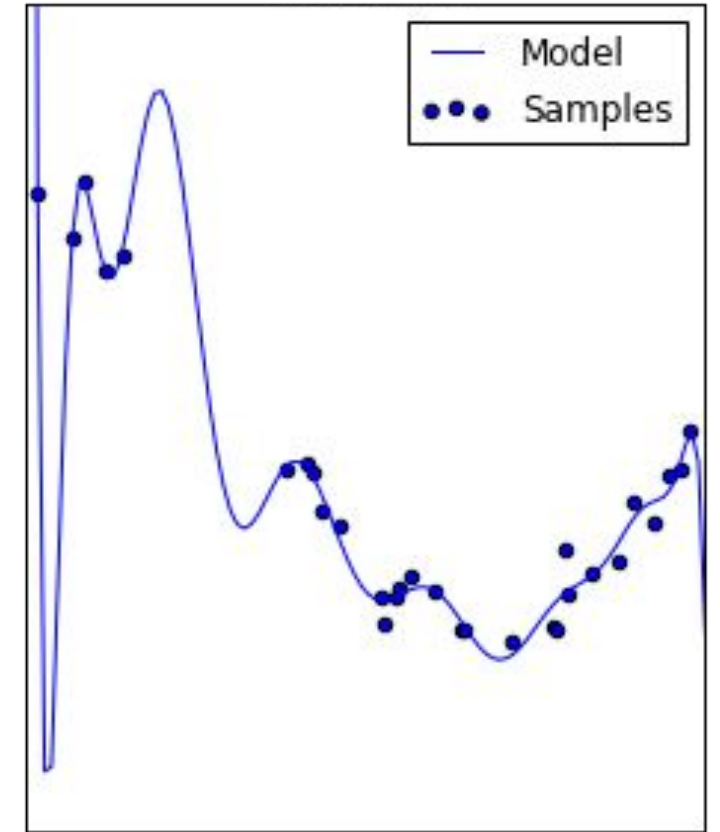
Underfitting



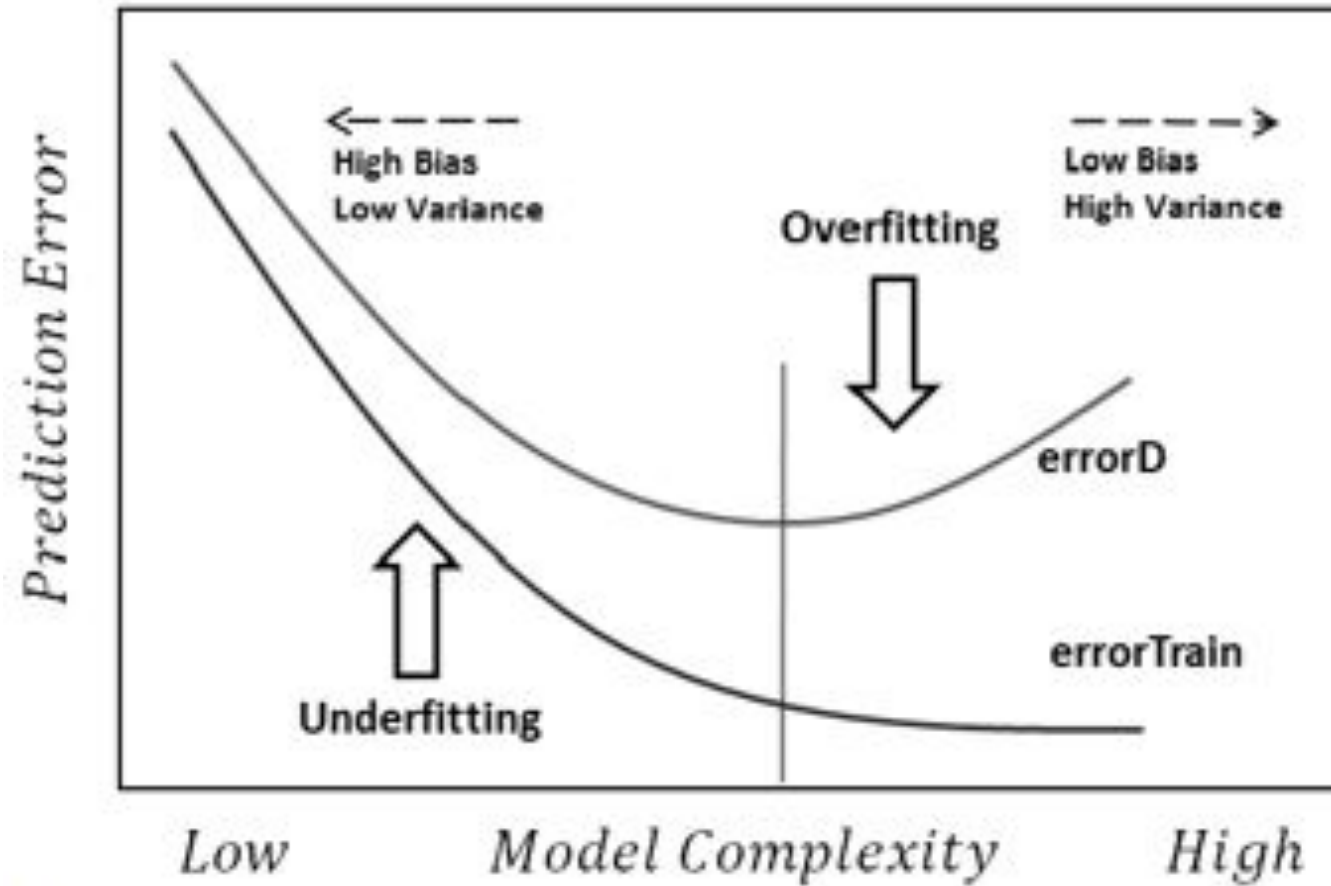
Proper generalization



Overfitting



Bias-Variance Tradeoff



Clustering vs Classification

Example to Differentiate

Suppose you have data about students' **study hours** and **marks**.

•Classification:

If you already know some students' results ("Pass" or "Fail"), you can **train a model** (like Logistic Regression) to predict whether a *new student* will pass or fail → **Supervised Learning**.

•Clustering:

If you **don't know the results**, you can use **K-Means Clustering** to group students based on their patterns (e.g., high performers, average performers, low performers) → **Unsupervised Learning**.

Clustering vs Classification

Basis	Clustering	Classification
Type of Learning	Unsupervised Learning	Supervised Learning
Data Labels	No predefined labels; the algorithm groups data based on similarity.	Labeled data is used; model learns from known categories or classes.
Objective	To discover hidden patterns or natural groupings in data.	To predict predefined categories for new data points.
Output	Groups (clusters) of similar data points — labels are generated by the algorithm.	Known class labels — model predicts which class a new point belongs to.

Basis	Clustering	Classification
Example Algorithms	K-Means, Hierarchical Clustering, DBSCAN	Logistic Regression, KNN, Decision Tree, SVM
Example Use Case	Group customers based on spending habits (e.g., high, medium, low spenders).	Predict whether a customer will buy a product (Yes/No).
Input Requirement	Only feature values (no target variable).	Requires both features (X) and labels (Y) for training.
Evaluation Metrics	Inertia, Silhouette Score, Davies-Bouldin Index	Accuracy, Precision, Recall, F1-score, ROC-AUC
Interpretation	Explores data structure and similarity.	Classifies data into known categories.
Real-life Example	Grouping students by learning behavior.	Predicting if a student will pass or fail.

Defining and Training Models

In machine learning, the workflow begins with **defining the model** and then **training** it on data.

Example (Linear Regression):

$$y = \beta_0 + \beta_1 x$$

Model parameters: β_0, β_1

Training the Model

Training means **finding the best parameters** by using the training dataset.

Evaluating Model Performance

Once a model is trained, we must evaluate how well it performs on unseen data.

A. Train–Test-Validation Split

The data is divided into:

- **Training set** : Usually **60–70%** of the data; Used to **train** the model
- **Validation set**: Usually **10–20%** of the data; Used to **tune hyperparameters** and select the best model
- **Testing set**: Usually **20–30%** of the data; Used **only at the end to evaluate** the **final model's performance** on unseen data

B. Evaluation Metrics

1. Regression Models

Used when output is continuous.

- **Mean Absolute Error (MAE)**

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE)**

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- **Root Mean Squared Error (RMSE)**

$$RMSE = \sqrt{MSE}$$

- **R² (Coefficient of Determination)**

Measures how much variation is explained by the model.

2. Classification Models

Used when output is categorical.

- **Accuracy**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision**

$$Precision = \frac{TP}{TP + FP}$$

- **Recall**

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score** (harmonic mean of Precision and Recall)

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

- **Confusion Matrix**

Table showing TP, TN, FP, FN.

- **ROC Curve & AUC**

Measures model's ability to discriminate classes.

- AUC close to 1 → great model
- AUC = 0.5 → random model

C. Cross-Validation

To ensure model generalizes well:

- Common: **k-fold cross-validation**
- Dataset is divided into k parts; each part used once for testing.

D. Bias–Variance Tradeoff

Performance is influenced by:

- High Bias** → underfitting
- High Variance** → overfitting

Goal: balance both.